

Server Manage 운영 위키

DECS 서버 관리 코드의 기능, 경계와 설계 의도

문서 기준: 2026-07-17 저장소 상태

소스: /home/jy/server_manage

목차

1. 전체 구조
2. container-images
3. kerberos-nfs
4. monitoring
5. remote-operations
6. server-state
7. user-lifecycle

Server Manage 운영 위키

원본: MANUAL.md

목차

디렉터리	핵심 기능	상세 매뉴얼
container-images/	CUDA/TensorFlow 이미지와 컨테이너 런타임	열기
user-lifecycle/	사용자·그룹·컨테이너·UID/GID·Kerberos 생명주기	열기
server-state/	서버 desired state와 점검·복구 계획	열기
remote-operations/	WOL, 부팅 점검과 컨테이너 재시작	열기
monitoring/	Prometheus/Grafana, exporter, 경보와 포렌식	열기
kerberos-nfs/	Kerberos/NFS 정책, 키탭과 복구 도구	열기

container-images 설계·운영 매뉴얼

원본: container-images/MANUAL.md

역할: DECS GPU 컨테이너 이미지의 빌드 입력, 시작 시 런타임 구성, 검증을 소유한다.

1. 책임 범위

`container-images` 는 여러 CUDA/TensorFlow 조합을 동일한 사용자 경험으로 제공한다. 이미지가 책임지는 것은 OS·CUDA·Python/Jupyter 패키지와 컨테이너 안의 계정, SSH, 선택적 VNC, Kerberos ccache 사용 환경이다.

이미지는 UID/GID를 결정하지 않는다. 사용자 identity, 포트, 홈 디렉터리와 Kerberos keytab은 `user-lifecycle` 와 호스트가 관리한다. 이 경계를 지켜야 이미지가 사용자 DB나 특정 서버에 종속되지 않는다.

2. 디렉터리 지도

경로	핵심 기능
<code>Dockerfile</code>	모든 variant가 공유하는 단일 이미지 정의
<code>image-variants.json</code>	base image, CUDA/TensorFlow, 최소 driver, alias의 권위 있는 목록
<code>entrypoint.sh</code>	실행 시 사용자·그룹·SSH·Jupyter·VNC·Kerberos 환경 구성
<code>scripts/variant_matrix.py</code>	manifest를 Docker/GitHub Actions build matrix로 변환
<code>scripts/build_variants.py</code>	로컬 build/push 명령 생성과 실행
<code>scripts/test_image_variants.py</code>	이미지 메타데이터, TensorFlow와 선택적 GPU smoke test
<code>scripts/test_uid_create_container.py</code>	<code>user-lifecycle</code> CLI와의 dry-run 계약 검증
<code>tests/</code>	root_squash, Kerberos, sudo와 build 설정 회귀 테스트
<code>.github/workflows/docker-publish.yml</code>	PR merge 또는 수동 실행 시 matrix build/push

3. 핵심 기능

3.1 manifest 기반 이미지 variant

`image-variants.json` 한 곳에 다음 항목을 둔다.

- variant ID와 base NVIDIA CUDA image
- CUDA, TensorFlow, Python, Ubuntu 버전

- TensorFlow 설치 패키지와 conda 패키지 제약
- 필요한 최소 NVIDIA driver
- `stable` 또는 `experimental` 지원 상태
- 날짜가 없는 사용자용 alias

현재 안정 계열은 CUDA 11.8, 12.2, 12.3, 12.5이며 CUDA 12.8/H200 계열은 실장비 검증 전까지 experimental이다. 빌드 도구와 GitHub Actions가 같은 manifest를 읽으므로 로컬과 CI의 variant가 달라지지 않는다.

3.2 단일 Dockerfile

Dockerfile은 `BASE_IMAGE`, `CUDA_VERSION`, `TENSORFLOW_VERSION`, `MIN_NVIDIA_DRIVER`, `CONDA_PACKAGES` 등을 build argument로 받는다. 모든 variant는 공통으로 SSH, Kerberos client, 한국어 글꼴/입력기, Chrome, Xfce, TigerVNC/noVNC, Miniforge, Jupyter를 포함한다.

variant별 Dockerfile을 복제하지 않은 이유는 보안 패치나 공통 패키지 변경을 한 번만 적용하고, 차이는 manifest의 데이터로 검토하기 위해서다. 버전별 예외가 필요하면 먼저 manifest 값으로 표현하고, 정말 다른 설치 흐름일 때만 Dockerfile에 조건을 추가한다.

3.3 실행 시 identity 구성

`entrypoint.sh`의 주요 흐름은 다음과 같다.

1. 이미지 variant와 실제 host driver 정보를 출력한다.
2. `USER_ID`, `UID`, `GID`, `USER_GROUP`을 검증하고 기존 홈의 owner와 맞춘다.
3. supplemental group과 sudo 정책을 구성한다.
4. SSH 로그인과 Jupyter 설정을 만든다.
5. Kerberos 모드이면 전달받은 ccache를 사용자 환경에 연결한다.
6. Jupyter를 시작하고, 요청된 경우에만 VNC/noVNC를 시작한다.

호스트 mount가 `root_squash`인 환경에서는 root가 사용자 홈을 임의로 `chown`할 수 없다. 그래서 이미 결정된 UID/GID로 사용자를 실행하고, 홈에 쓸 수 없는 root 작업을 최소화한다. 사용자 홈의 기존 비밀번호와 VNC password 파일도 재시작 때 보존한다.

3.4 Kerberos와 sudo 격리

Kerberos 활성화 시 keytab은 컨테이너에 전달하지 않는다. 호스트의 root-only service가 keytab으로 ticket을 갱신하고 `/run/user/<uid>/krb5cc`만 bind mount한다. 컨테이너에는 `KRB5CCNAME`과 기대 principal만 전달한다.

기본 `DECS_USER_SUDO_MODE=restricted`는 패키지 설치에 필요한 제한된 sudo는 허용하지만 UID 전환, mount, 권한 변경, root shell과 우회 가능한 interpreter 실행을 막는다. 이는 컨테이너 root가 다른 UID로 가장해 호스트의 다른 ccache를 사용하는 경로를 줄이기 위한 설계다.

사용자 그룹 공유는 관리자 sudo 대신 `group-dir-share` helper로 제공한다. 사용자 자신의 권한으로 디렉터리를 만들고 `2770`과 default ACL을 설정하므로 공유 기능과 identity 격리를 함께 유지한다.

3.5 GPU 호환성 표시와 강제 모드

이미지는 시작할 때 build에 기록된 최소 NVIDIA driver와 `nvidia-smi` 결과를 출력한다. 기본값은 경고 중심이며 `STRICT_CUDA_COMPAT=true` 일 때만 최소 driver보다 낮은 host에서 시작을 실패시킨다.

기본을 강제 실패로 두지 않은 이유는 GPU가 없는 검증 환경이나 긴급 진단 container까지 막지 않기 위해서다. 운영 배포에서 호환성을 반드시 보장해야 하면 strict mode를 명시한다.

4. 빌드와 배포 흐름

```
image-variants.json
|
+--> variant_matrix.py --> GitHub Actions matrix --> build/push
|
+--> build_variants.py -----> local build/push
|
+--> test_image_variants.py -----> smoke test
```

전체 명령 확인:

```
cd /home/jy/server_manage/container-images
python3 scripts/build_variants.py --dry-run
```

특정 variant build:

```
python3 scripts/build_variants.py \
  --variant cuda12.5-tf2.20-ubuntu22.04
```

registry push는 build 결과와 tag 목록을 확인한 뒤 `--push` 를 추가한다. 날짜 tag는 재현 가능한 배포 지점을 만들고, `stable` / `latest` alias는 현재 권장 variant를 가리킨다. 운영 기록에는 alias보다 날짜가 포함된 tag를 남기는 것이 좋다.

5. 사용자 생명주기와의 계약

`uidctl create-container` 는 image 이름과 version tag, 최종 UID/GID, 포트와 home mount를 Docker 환경변수/ 옵션으로 전달한다. 이미지가 기대하는 주요 값은 다음과 같다.

값	의미
<code>USER_ID</code>	컨테이너 사용자 이름
<code>UID, GID</code>	DB·AD·NFS와 이미 일치가 검증된 숫자 identity
<code>USER_GROUP</code>	primary group 이름

값	의미
ENABLE_VNC	VNC/noVNC opt-in
KRB5CCNAME	컨테이너에 보이는 ccache 경로
DECS_KERBEROS_HOST_KEYTAB	호스트 관리 ticket을 기다리는 모드
DECS_USER_SUDO_MODE	disabled, restricted, allowed 중 하나

TARGET_UID 나 TARGET_GID 같은 두 번째 identity 체계를 만들지 않는다. 하나의 UID / GID 만 사용해야 container, NFS와 DB가 어긋나는 오류를 조기에 발견할 수 있다.

6. 테스트 전략

빠른 정적/회귀 테스트:

```
cd /home/jy/server_manage/container-images
bash tests/test_image_build_config.sh
bash tests/test_entrypoint_root_squash.sh
```

이미지 smoke test:

```
python3 scripts/test_image_variants.py \
  --variant cuda12.5-tf2.20-ubuntu22.04
```

실제 GPU까지 확인할 때만 --gpu 를 사용한다. 사용자 생성 계약은 실제 DB를 바꾸지 않는 dry-run/print 경로로 검증한다.

```
python3 scripts/test_uid_create_container.py \
  --variant cuda12.5-tf2.20-ubuntu22.04 \
  --print-only
```

7. 변경 가이드

새 variant 추가

1. image-variants.json 에 버전, base image와 최소 driver를 추가한다.
2. python3 scripts/variant_matrix.py 결과의 tag를 확인한다.
3. build dry-run과 shell 회귀 테스트를 실행한다.
4. 실제 image smoke test와 가능하면 대상 GPU host test를 수행한다.

5. 검증 전에는 `support: experimental` 과 명시적인 alias를 사용한다.

공통 런타임 변경

`Dockerfile` 변경과 `entrypoint.sh` 변경을 구분한다. package/파일은 build 시점에 넣고, 사용자 UID나 mount처럼 컨테이너마다 달라지는 항목만 entrypoint 에서 처리한다. entrypoint 변경은 재시작 시 idempotent한지, 기존 홈의 파일을 덮어쓰지 않는지, root_squash 환경에서 동작하는지를 함께 확인한다.

8. 운영 안전 수칙

- 실제 사용자 비밀번호를 image layer나 manifest에 넣지 않는다.
- keytab을 image 또는 container mount로 제공하지 않는다.
- `latest` 만 기록하지 말고 장애 분석이 가능한 날짜 tag를 남긴다.
- experimental variant를 stable alias로 바꾸기 전에 실장비 GPU test를 한다.
- sudo 완화는 ccache와 host bind mount에 대한 우회 가능성을 먼저 검토한다.
- 로컬 source 변경과 이미 registry에 push된 image를 구분해서 진단한다.

kerberos-nfs 설계·운영 매뉴얼

원본: kerberos-nfs/MANUAL.md

역할: FARM/LAB Kerberos·NFS의 운영 정책, 적용 상태, 키탭 관리와 수동 복구 근거를 소유한다.

1. 이 디렉터리의 성격

`kerberos-nfs` 는 일반 애플리케이션 패키지가 아니라 **운영 snapshot**과 **제한된 helper의 집합**이다. NAS export, AD identity, service principal, NFS mount는 서로 다른 시스템에 분산되어 있어 코드만으로 현재 상태를 설명할 수 없다. 따라서 실행 가능한 도구와 함께 적용 상태, 검증 결과, 과거 장애 근거를 보존한다.

이 디렉터리는 사용자별 keytab 생성이나 컨테이너 생성 transaction을 소유하지 않는다. 그 흐름은 `user-lifecycle`에 있다. 서버 전체 desired state와 일반적인 client bootstrap은 `server-state` , 지속적인 GSS/NFS 관측은 `monitoring`이 소유한다.

2. 디렉터리 지도와 신뢰 수준

경로	분류	사용 원칙
<code>README.md</code>	개요	기본 정책과 전체 위치 확인
<code>INVENTORY.md</code>	분류표	active/reference/risky/PoC 여부를 먼저 확인
<code>APPLIED_STATE.md</code>	적용 상태	FARM의 현재 검증된 mount profile 기준
<code>bin/</code>	active helper	사용법과 대상 host를 확인한 뒤 실행
<code>bin/recovery/</code>	수동 복구	운영 영향 검토와 명시적 승인 후 실행
<code>config/</code>	예제/reference	실제 <code>/etc/krb5.conf</code> 와 차이를 검토
<code>docs/farm/</code>	runbook/증거	최종 runbook을 우선하고 낱짜 문서는 당시 증거로 해석
<code>systemd/user/</code>	로컬 자동화	NAS keytab watcher의 사용자 timer/service
<code>labs/lab-kerberos-poc/</code>	실험	운영 LAB 경로와 분리, 바로 production에 적용하지 않음
<code>reference/</code>	사본	실제 생성 소유자보다 우선하지 않음
<code>archive/</code>	과거 작업	현재 실행 경로로 사용하지 않음

`INVENTORY.md` 를 둔 이유는 shell script라는 형식만으로 현재 실행해도 되는 도구인지, 사고 당시 기록인지 구분할 수 없기 때문이다. 낱짜가 오래되었다는 이유만으로 삭제하지 않고 증거로 남기되, active surface를 명시한다.

3. 현재 운영 원칙

3.1 기본 보안 flavor

FARM과 향후 LAB Kerberos client의 기본은 `sec=krb5` 다. 인증은 Kerberos로 보장하되 RPCSEC_GSS integrity/privacy wrapping의 추가 비용을 피한다. `sec=krb5i` 와 `sec=krb5p` 는 integrity 또는 privacy 요구가 성능 비용보다 중요한 명시적 경로와 시험에 사용한다.

과거 PoC 문서의 `sec=krb5p` 기록은 당시 시험 결과이며 현재 기본값을 뜻하지 않는다. 운영 기준은 `APPLIED_STATE.md` 와 최종 FARM runbook을 우선한다.

3.2 FARM mount profile

현재 문서화된 FARM 경로는 다음 형태다.

```
nas.farm.decs.internal:/volume1/share
-> /home/tako<N>/share

nfs4 vers=4.0,sec=krb5,proto=tcp,hard,
timeo=600,retrans=2,rsiz=1048576,wsiz=1048576,
addr=100.100.100.120,_netdev,exec,nouser
```

Synology가 실제 transfer size를 128 KiB로 협상할 수 있으므로 요청한 rsize와 실효값이 다른 것 자체를 장애로 보지 않는다. `hard` mount는 일시적인 NAS 장애에 데이터 오류를 반환하지 않는 대신 process가 D-state에 머물 수 있다. 이 때문에 timeout을 건 명령, readiness 순서, 포렌식이 별도로 필요하다.

3.3 identity 불변 조건

한 사용자에게 대해 다음 값이 같아야 한다.

```
UID DB ubuntu_uid
= AD RFC2307 uidNumber
= NFS client에서 보이는 home owner UID
= container UID

primary GID도 같은 원칙 적용
```

이 일치를 확인하지 않은 채 `chown` 으로 증상만 고치면 NAS internal mapping, AD object와 DB가 더 어긋날 수 있다. identity 복구는 `farm-kerberos-identity-flow-policy` 와 RFC2307 recovery 문서를 함께 본다.

4. 핵심 기능

4.1 NAS NFS service keytab watcher

`bin/watch-farm-nas-nfs-keytab.sh`는 FARM AD의 전용 계정 `svc-nfs-farm`과 NAS의 `/etc/krb5.keytab`, `/etc/nfs/krb5.keytab`을 비교한다. AD KVNO, 필요한 NFS principal, 현재 ticket 복호화 가능성을 점검하고 필요하면 새 keytab을 export 병합한 뒤 NAS에 원자적으로 교체한다.

```
# 변경 없이 점검
./bin/watch-farm-nas-nfs-keytab.sh --check --no-restart

# drift를 복구하되 GSS 재시작 여부는 영향 검토
./bin/watch-farm-nas-nfs-keytab.sh --repair
```

watcher의 repair와 AD password rotation은 다른 작업이다. timer는 drift를 수리할 수 있지만 AD 암호를 바꾸지 않는다. 자동 watcher가 key rotation까지 수행하면 일시적 네트워크 실패가 전체 NFS service key 변경으로 확대될 수 있기 때문이다.

4.2 계획된 service key rotation

`bin/rotate-farm-nfs-service-key.sh`는 계정과 SPN 소유권을 먼저 확인한다. 실제 회전은 `--rotate --apply` 두 옵션을 모두 요구한다.

```
./bin/rotate-farm-nfs-service-key.sh --check
./bin/rotate-farm-nfs-service-key.sh --rotate --apply
```

회전은 새 random password, AD replication, 강제 watcher repair, NAS 적용이 한 묶음이다. 중간 실패 시 이전/새 KVNO와 NAS keytab의 principal을 각각 확인해야 한다. 유지보수 창과 활성 client 영향 확인 없이 실행하지 않는다.

4.3 FARM mount 복구

`bin/remount-farm-user-share-krb.sh`는 현재 기본 `sec=krb5` profile로 FARM share를 복구하기 위한 helper다. mount 변경은 실행 중인 container와 user session에 영향을 주므로, 먼저 다음 chain을 확인한다.

1. `/etc/krb5.keytab` 존재와 machine principal
2. machine `kinit -k`
3. NFS service `kvno`
4. `rpc_pipefs`와 `rpc-gssd`
5. 기존 mount와 D-state process
6. 대상 host를 제한한 remount

일반적인 기존 host 점검에는 `server-state`의 `kerberos_nfs_client_recovery.yml` check mode를 먼저 사용할 수 있다.

4.4 복구 helper

`bin/recovery/`에는 NAS export 직접 수정, 잘못된 management-IP mount 제거, legacy `sec=sys` mount 제거, home artifact owner 수리 도구가 있다. 이름에 `recovery`가 붙었다는 사실을 실행 허가해 해석하면 안 된다. 대상과 현재 상태를 읽기 전용 명령으로 확인하고 script 내용을 검토한 뒤 사용한다.

특히 NAS export를 쓰는 helper는 설정 파일을 직접 변경하므로 backup, diff, NAS 서비스 반영 방법과 되돌리기 절차가 있어야 한다.

5. keytab과 ticket 모델

NAS service identity

- AD의 전용 service account가 NFS SPN을 소유한다.
- NAS는 NFS service key가 포함된 root-only keytab을 가진다.
- watcher는 현재 AD KVNO와 NAS keytab을 검증한다.
- GSS service 재시작은 key 반영이 필요할 때만 수행한다.

사용자 identity

- 사용자 keytab 생성과 target host 설치에 `user-lifecycle`가 소유한다.
- keytab은 `/etc/decs-krb/keytabs/<user>.keytab`에 root-only로 둔다.
- systemd refresh가 `/run/user/<uid>/krb5cc`를 만든다.
- container에는 keytab이 아니라 ccache만 bind mount한다.

`reference/decs-krb-refresh`는 이 refresh logic의 참고 사본이다. 실제 생성 logic을 수정할 때는 `user-lifecycle/script/uid_manager/kerberos/`를 변경한다.

6. LAB PoC의 위치

`labs/lab-kerberos-poc/`는 Samba AD, NFSv4.1, 다양한 kernel/client 조합을 격리해 재현하기 위한 실험 영역이다. 운영 LAB share와 다른 export나 VM을 사용할 수 있으며, 결과 디렉터리는 성공·실패 당시의 증거다.

최근 VM slot regression 도구는 다음을 분리한다.

- VM provisioning과 isolated network
- AD/NFS server/client 구성
- kernel matrix 전환
- workload와 fault injection
- client/server capture와 결과 분석

PoC에서 성공한 설정도 운영 topology, idmap domain, 기존 mount와 사용자 영향을 검토한 뒤 `server-state`나 정식 playbook으로 승격해야 한다.

7. 다른 모듈과의 경계

작업	소유 모듈	이 디렉터리의 역할
사용자 AD/keytab/home 준비	user-lifecycle	정책·복구 근거 제공
host 공통 Kerberos client baseline	server-state	현재 mount/runbook 기준 제공
GSS readiness/canary/포렌식	monitoring	장애 해석과 운영 정책 제공
container ccache 소비	container-images	보안 모델 제공
NAS service key와 export	kerberos-nfs	직접 소유

동일한 shell 조각을 여러 모듈로 복사하지 말고, 위험한 NAS 작업은 이 디렉터리의 명시적 helper/runbook으로 되돌아오게 한다.

8. 장애 진단 순서

1. `findmnt` 로 source, target, `sec`, version과 실제 option을 확인한다.
2. `klist -k` 와 `kinit -k` 로 host keytab을 확인한다.
3. `kvno nfs/<storage-fqdn>@<REALM>` 으로 service ticket을 확인한다.
4. `rpc_pipefs`, `rpc-gssd`, journal 상태를 확인한다.
5. 사용자 ccache와 `klist` 를 해당 UID 권한으로 확인한다.
6. DB·AD RFC2307·NFS owner 숫자를 비교한다.
7. D-state와 mount responsiveness는 `monitoring` 지표와 포렌식 snapshot을 확인한다.
8. 공유 service 재시작이나 remount는 마지막 단계로 남긴다.

9. 문서 유지 규칙

- 새 helper는 `INVENTORY.md` 에 active/reference/risky 상태를 추가한다.
- 실제 production profile이 바뀌면 `APPLIED_STATE.md` 에 검증 시각과 대상을 기록한다.
- 실험 결과는 current default와 당시 실험 조건을 명확히 구분한다.
- keytab, 암호, local env, incident 개인정보와 대용량 capture를 커밋하지 않는다.
- 최종 runbook과 과거 rollout 문서가 충돌하면 최종 runbook과 적용 상태를 우선하고, 충돌 사실을 문서에 남긴다.

monitoring 설계·운영 매뉴얼

원본: monitoring/MANUAL.md

역할: FARM/LAB 서버와 사용자 GPU workload의 지속 관측, 시각화, 경보와 제한된 자동 복구를 소유한다.

1. 책임 범위

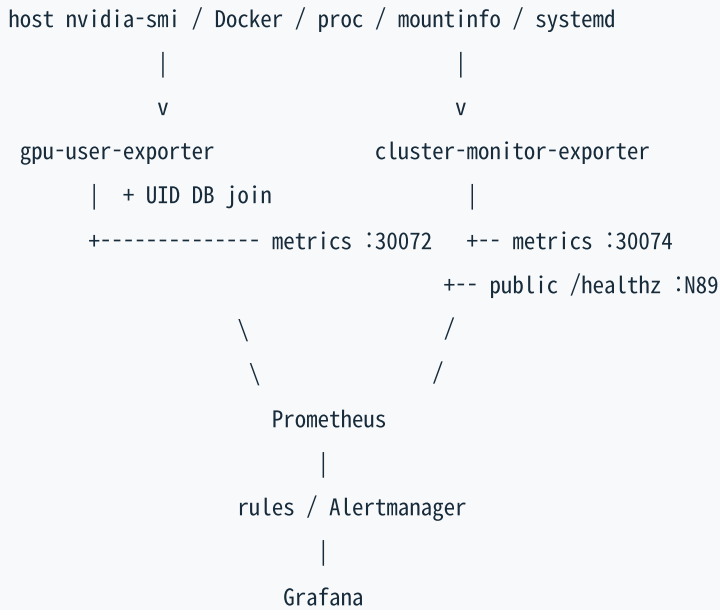
`monitoring`은 Kubernetes의 Prometheus/Grafana 자원과 각 GPU host에서 실행되는 exporter를 함께 관리한다. 서버가 켜진 직후 한 번 실행하는 점검은 `remote-operations`, 서버에 무엇이 설치되어야 하는지는 `server-state`가 소유한다. 이 디렉터리는 **실행 중인 시스템을 반복 관측하는 경로**다.

지속 관측과 부팅 작업을 분리한 이유는 일시적인 부팅 지연을 장기 장애로 오판하지 않고, exporter 재배포가 WOL이나 container restart 순서에 영향을 주지 않게 하기 위해서다.

2. 디렉터리 지도

경로	핵심 기능
<code>farm/prometheus/</code>	FARM kube-prometheus-stack values, PV, storage class
<code>lab/prometheus/</code>	독립 LAB Prometheus와 PV 설정
<code>farm/gpu/</code> , <code>lab/gpu/</code>	NFD와 NVIDIA device plugin 설정
<code>exporters/cluster-monitor-exporter/</code>	mount, GPU, Docker, container, 연결성, NFS GSS 지표
<code>exporters/gpu-user-exporter/</code>	GPU process를 UID DB의 실제 사용자/container에 귀속
<code>ansible_playbook/</code>	exporter, GSS health, NFS forensics 배포 entry point
<code>grafana/dashboards/</code>	FARM/LAB GPU, NIC, storage latency dashboard
<code>grafana/datasources/</code>	LAB Prometheus datasource override
<code>nfs-forensics/</code>	NFS incident용 제한된 packet/kernel trace ring
<code>shared/gpu-stress-test/</code>	공통 GPU test workload

3. 전체 데이터 흐름



FARM Grafana가 FARM과 LAB dashboard를 함께 제공하므로 dashboard는 cluster 하위가 아니라 공통 grafana/에 둔다. LAB dashboard는 UID가 prometheus-lab 인 datasource를 기대한다.

4. cluster-monitor-exporter

4.1 관측 대상

Go exporter는 host에서 다음 상태를 수집한다.

- 필요한 mount의 source/target 일치와 응답성
- storage peer 연결성과 mount failure 진단
- /proc 의 blocked process와 NFS/RPC D-state
- host NVIDIA GPU 상태
- Docker daemon과 대상 container 상태
- container SSH/GPU readiness와 NVML mismatch
- 외부 인터넷/heartbeat 연결성
- Kerberos NFS readiness, canary와 bounded recovery 상태
- NFS 포렌식 snapshot 상태

/metrics 는 마지막 collection 결과를 제공하고 /healthz 는 collection이 최근에 성공했는지 확인한다. 별도 public health listener는 서버 번호별 예약 N89 포트로 외부에서 host reachability를 확인하게 한다.

4.2 자체 renderer와 명령 timeout

exporter는 수집 결과를 Prometheus text format으로 렌더링한다. 외부 명령은 timeout을 두고 실행한다. NFS `hard` mount 장애에서 child가 D-state가 되면 일반적인 context cancel만으로 종료되지 않을 수 있기 때문에, 수집 loop가 무한히 막히지 않도록 process와 결과 보존을 방어적으로 처리한다.

collection freshness도 health 판단에 포함한다. HTTP server가 응답한다는 사실만으로 수집이 정상이라고 보지 않기 위해서다.

4.3 제한된 복구

exporter는 설정에 따라 멈춘 대상 container 시작, container SSH 시작, 안전한 NVML library mismatch 복구처럼 범위가 제한된 작업을 할 수 있다. Kerberos NFS mount 복구는 별도 timer/script 상태를 관측하며 다음 gate를 통과한 missing mount만 복구한다.

1. fstab에 기대한 항목이 있다.
2. DNS와 storage RPC가 응답한다.
3. Kerberos readiness chain이 통과한다.
4. 해당 환경에서 요구되는 canary가 통과한다.
5. retry/inhibit 상태가 복구를 허용한다.

공유 NAS 서비스 재시작, 임의 unmount, user share 데이터 읽기는 exporter의 책임이 아니다. 자동 복구가 장애 원인보다 더 큰 장애 반경을 만들지 않게 하는 경계다.

5. gpu-user-exporter

5.1 해결하는 문제

`nvidia-smi` 는 GPU process와 PID를 보여 주지만 DECS 사용자 이름이나 DB의 container owner를 직접 알지 못한다. exporter는 다음 정보를 조합한다.

1. `nvidia-smi` 로 GPU device, process memory와 pmon utilization을 읽는다.
2. host `/proc/<pid>/cgroup` 에서 Docker container ID를 찾는다.
3. Docker inspect로 container 상태와 device 할당을 확인한다.
4. UID DB의 활성 container record를 cache하고 실제 사용자/username과 join한다.
5. 사용자·container·GPU별 memory, utilization, process count를 집계한다.

주요 지표는 `docker_gpu_user_memory_used_bytes` , `docker_gpu_user_sm_utilization_percent` , `docker_gpu_user_process_count` , device 전체 사용률과 exporter scrape/DB cache 상태다. DB에서 찾지 못한 process는 임의 사용자에게 귀속하지 않고 ignored count로 노출한다.

5.2 DB cache 설계

매 scrape마다 DB와 Docker에 과도한 부하를 주지 않도록 활성 container owner 목록을 일정 간격으로 cache한다. DB refresh 성공 여부와 cache entry 수를 별도 지표로 내보내므로 stale mapping 가능성을 운영자가 확인할 수 있다. DB 비밀번호는 배포 시 local env에서 target systemd 환경 파일로 전달하며 소스나 dashboard에 넣지 않는다.

6. Prometheus와 Grafana

FARM

- `kube-prometheus-stack` 을 사용한다.
- Prometheus/Grafana local PV는 관리 desktop의 `/data` 경로를 사용한다.
- Grafana NodePort 기본은 `30080` 이다.
- FARM/LAB datasource와 dashboard를 한 Grafana에서 제공한다.

LAB

- LAB8 control plane의 독립 cluster로 운영한다.
- LAB Prometheus는 기본적으로 cluster 내부 service다.
- FARM Grafana가 질의해야 할 때만 검토된 routed endpoint를 제공한다.

cluster별 values를 나눈 이유는 target, retention, storage, Alertmanager routing과 control plane 장애가 독립적이기 때문이다. dashboard는 공통 UI이므로 한 곳에 두되 datasource UID와 label로 FARM/LAB를 분리한다.

7. NFS GSS health와 forensics

Readiness와 canary

GSS health 배포는 다음 순서를 검증한다.

```
keytab -> kinit -> NFS service kvno -> rpc_pipefs -> rpc-gssd -> canary
```

부팅 시 mount가 host identity 준비보다 먼저 실행되는 race를 막기 위해 단계별 결과를 상태 파일에 기록한다. canary는 실제 user share가 아니라 전용 read-only 경로를 사용하는 것이 원칙이다. 전용 export가 없는 FARM은 canary gate를 비활성화하되 나머지 readiness는 유지한다.

Passive forensics

`nfs-forensics` 는 TCP/2049 packet의 크기 제한 ring과 저용량 ftrace instance, 5초 간격 상태 관측을 유지하다 incident threshold에서 snapshot을 보존한다. 다음 작업은 하지 않는다.

- mount/unmount
- 사용자 share 읽기
- NFS/GSS service restart
- recovery 또는 reboot

`sec=krb5` 는 payload를 암호화하지 않으므로 capture와 incident directory는 root-only로 유지한다. exporter는 `/run/decs-nfs-forensics/status.json` 을 읽을 뿐 capture service를 제어하지 않는다. 관측 경로가 장애 복구와 결합되어 증거를 파괴하지 않게 하기 위한 설계다.

8. 배포

두 exporter를 한 host에 배포:

```
cd /home/jy/server_manage/monitoring
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \
ansible-playbook ansible_playbook/deploy_exporters.yml \
-e exporter_hosts=farm8
```

cluster-monitor exporter만 전체 배포:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \
ansible-playbook ansible_playbook/deploy_cluster_monitor_exporter_all.yml
```

NFS GSS health를 제한된 host에 배포:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \
ansible-playbook ansible_playbook/deploy_nfs_gss_health.yml \
-e nfs_gss_health_hosts=lab8
```

NFS forensics 배포:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \
ansible-playbook ansible_playbook/deploy_nfs_forensics.yml
```

playbook은 controller에서 Go binary를 build하고 systemd env/service를 target에 설치한 뒤 local HTTP endpoint를 검증한다. 새 서버는 Ansible inventory, Prometheus scrape target, 공개 health port와 dashboard label을 모두 확인한다.

9. 테스트와 검증

```
cd /home/jy/server_manage/monitoring

go test ./exporters/cluster-monitor-exporter/cmd/cluster-monitor-exporter/...
go test ./exporters/gpu-user-exporter/...
bash nfs-forensics/tests/test_watch.sh
bash exporters/cluster-monitor-exporter/tests/test_nfs_gss_ready.sh
bash exporters/cluster-monitor-exporter/tests/test_nfs_gss_mount_recovery.sh
```

배포 후 최소 확인 항목:

- `:30072/-/healthy` 와 `:30072/metrics`
- `:30074/healthz` 와 `:30074/metrics`
- public `N89/healthz`
- Prometheus target의 up 상태와 scrape freshness
- Alert rule label이 Alertmanager route와 일치하는지
- FARM/LAB dashboard가 올바른 datasource를 보는지

10. 변경 가이드와 안전 수칙

- 새 metric은 이름, help, label cardinality와 실패 시 값을 함께 설계한다.
- container ID, PID처럼 cardinality가 큰 label은 필요성을 검토한다.
- 자동 복구는 사전조건, timeout, retry limit와 inhibit 상태를 가져야 한다.
- NFS probe는 user data를 읽지 않는 경로와 bounded timeout을 사용한다.
- exporter binary와 runtime incident data를 Git에 추가하지 않는다.
- local datasource credential과 DB password는 example 파일에 실제 값으로 넣지 않는다.
- 지속 monitor에서 새로 발견한 high-risk 복구는 `kerberos-nfs` 또는 소유 모듈의 명시적 runbook으로 분리한다.

remote-operations 설계·운영 매뉴얼

원본: remote-operations/MANUAL.md

역할: FARM/LAB 서버를 깨우고, 부팅 시 한 번 상태를 확인한 뒤 기존 컨테이너를 시작한다.

1. 책임 범위

remote-operations 는 management host의 remote-boot.service 가 실행하는 부팅 orchestration이다. Wake-on-LAN, 우선 서버 gate, 1회 host health check와 container post-check를 소유한다.

주기적인 mount/GPU/container 관측은 monitoring 에 있다. 이 경계를 나눈 이유는 부팅 시 복구와 상시 self-healing이 서로 다른 retry 정책과 장애 의미를 가지기 때문이다. 이 디렉터리에서 관리하는 systemd unit은 remote-boot.service 하나다.

2. 디렉터리 지도

경로	핵심 기능
script/run_remote_boot.sh	전체 부팅 흐름의 entry point
script/wake_targets.sh	target 선택과 WOL magic packet 전송
script/wait_for_priority_servers.sh	priority/remaining host health gate
script/check_server_boot_health.sh	mount, GPU, 임시 test container 점검
script/restart_all_remote_containers.sh	기존 container 시작과 SSH/GPU post-check
script/create_test_container.sh	health check용 임시 GPU container 생성
script/delete_test_container.sh	임시 container 정리
script/common.sh	config, target, Ansible, log, alert 공통 함수
script/install_remote_boot_service.sh	systemd unit 설치·활성화
script/dry_run_remote_boot.sh	WOL/health/container/full flow simulation
script/integration_smoke_test.sh	수동 Ansible/Docker/GPU 통합 확인
config/remote_boot.example.env	공개 가능한 설정 구조
config/remote_boot.local.env	실제 MAC·webhook 등을 담는 ignored 설정

3. 부팅 상태 머신

```
pre-delay
|
v
wake priority targets
|
v
priority health gate ---- failure ----> stop + alert
|
v
secondary delay
|
v
wake remaining targets
|
v
remaining health gate -- failure -----> record + alert
|
v
start stopped target containers
|
v
per-container SSH/GPU post-check
```

priority server를 먼저 깨우는 이유는 storage, DB, control plane처럼 나머지 서버가 의존하는 기반을 먼저 검증하기 위해서다. gate를 끌 수는 있지만 기본은 우선 대상의 health가 통과한 뒤 나머지를 깨우는 방식이다.

4. 핵심 기능

4.1 target 정규화

설정은 FARM/LAB 전체 목록, 기본 대상과 priority 대상을 별도로 가진다. 명령행 target이 있으면 기본 대상을 덮어쓴다. `all`, domain group, 개별 `FARM1` / `LAB10` 을 동일한 parser로 해석하여 중복을 제거한다.

MAC address와 broadcast IP는 WOL에만 쓰고, 상태 확인과 원격 명령은 Ansible inventory를 사용한다. 네트워크 주소와 논리 server ID를 분리하면 SSH port나 주소가 바뀌어도 workflow 이름을 유지할 수 있다.

4.2 1회 host health check

`check_server_boot_health.sh` 는 target에 대해 다음을 확인한다.

- Ansible/SSH 도달성
- 필수 FARM/LAB share mount

- host `nvidia-smi`
- 임시 test container 생성
- container 내부 GPU와 필요한 home mount
- test 종료 후 container 삭제

실제 사용자 container 대신 고정된 health test identity와 image를 사용하는 이유는 특정 사용자 상태를 서버 health로 오판하지 않고, test artifact를 명확히 정리하기 위해서다.

4.3 기존 container 재시작과 post-check

선택된 서버의 stopped target container를 시작한 뒤 각 container의 SSH와 GPU를 제한 시간 동안 확인한다. post-check 실패는 container별로 기록하고 전체 stage 실패 정보에 포함한다.

container를 무조건 재생성하지 않는 이유는 사용자 홈과 DB record, 고정 포트, 실행 옵션의 권위가 `user-lifecycle`에 있기 때문이다. 부팅 모듈은 기존 container를 시작하고 준비 여부만 확인한다.

4.4 alert suppression과 로그

실패는 구성된 경우 내부 notify API를 통해 Slack으로 보내고, 그렇지 않으면 stub log에 남긴다. 동일 실패의 반복 알림을 줄이기 위한 alert state가 별도 디렉터리에 저장된다. `reset_remote_boot_alert_state.sh`는 이 suppression 상태만 초기화한다.

service log, host health log와 alert state를 나눈 이유는 실행 이력과 알림 deduplication 상태를 독립적으로 보존하기 위해서다.

5. 설정 모델

설정은 `remote_boot.example.env`를 복사하여 local 파일에서 관리한다.

```
cd /home/jy/server_manage/remote-operations
cp config/remote_boot.example.env config/remote_boot.local.env
```

주요 설정 그룹:

그룹	예
target	FARM/LAB 목록, 기본/priority target
순서/gate	pre-delay, gate timeout/poll, secondary delay
container	restart enable, timeout, post-check poll
network	Ansible inventory, broadcast IP, MAC address
health	필수 mount와 host share template
test container	image, UID/GID, mount, memory, runtime

그룹	예
logging/alert	log path, rotate count, notify API와 webhook

실제 MAC, password와 webhook은 `remote_boot.local.env`에만 둔다. example에는 변수 이름과 안전한 placeholder만 유지한다.

6. 사용법

target 확인과 수동 WOL:

```
cd /home/jy/server_manage/remote-operations
./script/wake_targets.sh --list-targets
./script/wake_targets.sh FARM1 LAB1
```

한 서버의 boot health:

```
./script/check_server_boot_health.sh --server-id FARM1
```

한 서버의 기존 container 시작/post-check:

```
./script/restart_all_remote_containers.sh FARM1
```

systemd 설치와 확인:

```
./script/install_remote_boot_service.sh
sudo systemctl start remote-boot.service
systemctl status remote-boot.service
journalctl -u remote-boot.service -b
```

7. dry-run과 검증

```
./script/dry_run_remote_boot.sh wake FARM1 LAB1
./script/dry_run_remote_boot.sh health FARM1
./script/dry_run_remote_boot.sh containers FARM1
./script/dry_run_remote_boot.sh full FARM1 LAB1
```

dry-run은 sleep, WOL, Ansible 변경과 Docker 작업 대신 계획을 기록한다. 설정 파싱, target 분할과 단계 순서를 production 영향 없이 검증하기 위한 공개 계약이다.

실제 연결을 확인할 때:

```
./script/integration_smoke_test.sh --scope priority
```

운영 전에는 최소한 다음을 확인한다.

- priority target이 실제 의존 순서와 맞는지
- MAC/broadcast IP와 Ansible host가 같은 물리 서버를 가리키는지
- health test image가 target GPU driver와 호환되는지
- 임시 container 이름이 실제 사용자 container와 충돌하지 않는지
- gate timeout이 정상 부팅 시간보다 충분한지
- 실패 알림에 비밀값이 포함되지 않는지

8. 실패 처리 원칙

- priority gate 실패 시 나머지 서버를 깨우지 않는 것이 기본이다.
- remaining server 일부 실패는 성공한 서버의 후속 작업과 실패 대상을 구분해 기록한다.
- test container는 성공/실패와 무관하게 정리를 시도한다.
- mount 또는 GPU 문제를 부팅 script에서 무제한 복구하지 않는다. 반복 상태는 **monitoring**으로 넘기고, Kerberos/NFS 고위험 복구는 해당 runbook을 따른다.
- 실제 사용자 container의 DB record나 Docker 옵션을 부팅 script에서 다시 만들지 않는다.

9. 새 서버 추가

1. **user-lifecycle/server_info/servers.jsonl** 과 Ansible inventory에 서버를 등록한다.
2. local env의 FARM/LAB target 목록과 MAC을 추가한다.
3. 필요한 경우 priority 여부와 필수 mount template를 정한다.
4. WOL dry-run과 개별 WOL을 확인한다.
5. 개별 boot health check를 실행한다.
6. container restart를 한 서버에 제한해 검증한다.
7. **monitoring**의 scrape target/exporter 배포를 별도로 완료한다.

10. 설계상 지켜야 할 경계

- systemd unit을 추가하기 전에 부팅 1회 책임인지 지속 monitor 책임인지 구분한다.
- 공통 shell helper를 수정하면 모든 stage의 dry-run과 alert redaction을 확인한다.
- 실제 secret을 example 또는 log에 쓰지 않는다.
- retry를 늘리는 것으로 storage/GSS 장애를 감추지 않는다.

- container 생성/삭제 비즈니스 규칙은 `user-lifecycle` CLI를 통해 수행한다.

server-state 설계·운영 매뉴얼

원본: server-state/MANUAL.md

역할: 서버별 원하는 상태를 정의하고, 실제 소유 모듈의 점검·복구 경로를 dry-run 계획으로 연결한다.

1. 왜 별도 조정 계층이 필요한가

GPU 서버 한 대에는 Docker, NVIDIA driver/toolkit, Kubernetes node, Kerberos NFS, exporter와 사용자 container 전제조건이 함께 필요하다. 그러나 이 기능을 하나의 거대한 playbook에 넣으면 소유권, 위험도와 변경 주기가 사라진다.

`server-state` 는 다음 두 질문만 담당하는 얇은 desired-state 계층이다.

1. 이 서버에 무엇이 있어야 하는가?
2. 점검 또는 복구는 어느 모듈의 어떤 명령이 소유하는가?

현재 구현은 의도적으로 dry-run 중심이다. 로컬 inventory와 profile을 읽고 remote check와 remediation command를 출력하지만 production host에 접속하거나 변경하지 않는다. `apply --execute` 는 구현되지 않았고 명시적으로 거부된다.

2. 디렉터리 지도

경로	핵심 기능
<code>bin/server-state</code>	Python CLI wrapper
<code>server_state/cli.py</code>	<code>list-hosts</code> , <code>check</code> , <code>plan</code> , <code>apply</code> 출력
<code>server_state/inventory.py</code>	공용 JSONL inventory 로드와 host 선택
<code>server_state/profiles.py</code>	YAML profile/catalog 파싱, set 확장과 template render
<code>config/profiles.yml</code>	desired-state check/remediation의 권위 있는 정의
<code>ansible_playbook/bootstrap_gpu_server.yml</code>	공통 GPU host baseline의 tag 기반 playbook
<code>ansible_playbook/kerberos_nfs_client_recovery.yml</code>	기존 host GSS readiness/recovery 순서
<code>docs/module-boundaries.md</code>	모듈별 소유권 경계
<code>docs/standard-gpu-server-pipeline.md</code>	신규/기존 서버 표준 단계
<code>tests/</code>	inventory와 profile expansion unit test

3. 데이터 모델

3.1 서버 inventory

기본 inventory는 `user-lifecycle/server_info/servers.jsonl` 이다. `server-state` 가 별도 서버 목록을 만들지 않는 이유는 주소, SSH port와 논리 server ID의 중복 권위를 피하기 위해서다.

loader는 FARM/LAB host 이름과 JSONL 필드를 정규화하고 다음 선택 방식을 제공한다.

- `all`
- `farm`, `lab` domain
- `farm8`, `FARM8` 같은 개별 host/server ID
- comma 또는 공백으로 구분한 여러 selector

3.2 profile catalog

`config/profiles.yml` 은 profile과 profile set을 정의한다. 각 profile은 소유 모듈, read-only check와 remediation을 가진다.

check kind:

kind	동작
<code>local-inventory</code>	inventory 존재를 로컬에서 확인
<code>local-path</code>	저장소 또는 controller 경로 존재 확인
<code>template-value</code>	서버별 render 값 존재 확인
<code>remote-read</code>	실행하지 않고 읽기 전용 원격 명령을 표시

remediation mode:

mode	출력
<code>command</code>	<code>DRY-RUN</code> 명령과 safety level
<code>manual</code>	<code>MANUAL</code> 상태와 소유 runbook/reference

profile set은 여러 profile을 정해진 순서로 확장한다. 신규 서버와 기존 서버가 같은 작은 profile을 재사용하므로 전역 설정이 한쪽 pipeline에서 빠지는 것을 줄인다.

4. 표준 GPU 서버 pipeline

순서	profile	소유자	목적
1	<code>baseline-host</code>	server-state	inventory, SSH, sudo, hostname preflight

순서	profile	소유자	목적
2	os-common	server-state	apt, NFS, Kerberos, network 도구
3	docker-engine	server-state	Docker와 systemd cgroup driver
4	nvidia-driver	server-state	host driver와 package hold
5	nvidia-container-runtime	server-state	Docker/containerd NVIDIA toolkit
6	kubernetes-node	server-state	kubeadm/kubelet/kubectl와 join gate
7	network-tuning	server-state	storage NIC RX queue 영속화
8	kerberos-nfs-client	kerberos-nfs	realm, keytab, GSS와 mount profile
9	monitoring-exporters	monitoring	cluster/GPU user exporter
10	user-container-host	user-lifecycle	lifecycle 실행 전제조건

new-host-bootstrap 와 existing-host-drift 가 이 profile들을 각각 신규 설치와 기존 상태 점검 관점에서 조합한다. managed-host 는 기존 관리 서버의 기본 alias다.

5. 모듈 소유권

server-state가 직접 소유하는 것

- 공통 OS package와 repository
- Docker engine의 systemd cgroup 설정
- NVIDIA driver/toolkit 설치 형태
- Kubernetes node package 전제조건
- storage NIC RX queue tuning
- profile/host 선택과 상태 vocabulary

다른 모듈에 위임하는 것

영역	실제 소유자	server-state의 행동
exporter와 dashboard	monitoring	배포 playbook 명령을 계획
NAS/AD/Kerberos 정책	kerberos-nfs	manual/gated runbook 참조
사용자·컨테이너 transaction	user-lifecycle	inventory를 읽고 CLI/playbook을 계획
WOL·boot sequence	remote-operations	controller 설정과 service install 점검
DECS image	container-images	필요 profile에서 image test를 참조

조정 계층이 다른 모듈의 구현을 복제하지 않는 이유는 한쪽 수정이 다른 복사본에 반영되지 않는 drift를 막고, 고위험 작업의 승인 gate를 유지하기 위해서다.

6. CLI 사용법

host 목록:

```
cd /home/jy/server_manage
./server-state/bin/server-state list-hosts --hosts all
./server-state/bin/server-state --format json list-hosts --hosts farm
```

점검 계획:

```
./server-state/bin/server-state check \
  --hosts farm8 \
  --profile managed-host
```

복구 계획:

```
./server-state/bin/server-state plan \
  --hosts farm8 \
  --profile monitoring-exporters
```

신규 서버 전체 순서:

```
./server-state/bin/server-state plan \
  --hosts farm8 \
  --profile new-host-bootstrap
```

현재 **apply** 도 같은 dry-run plan만 출력한다.

```
./server-state/bin/server-state apply \
  --hosts farm8 \
  --profile existing-host-drift
```

자동화가 결과를 소비할 때는 **--format json** 을 사용한다. text 출력 parsing에 의존하지 않기 위해 구조화 출력을 별도로 제공한다.

7. 안전 모델

상태 vocabulary는 다음 의미를 가진다.

상태	의미
OK	로컬에서 확인 가능한 조건 충족
MISSING	로컬 파일/값 없음
DRY-RUN	실행할 원격 check 또는 remediation 표시
MANUAL	자동화하지 않은 고위험/도메인 소유 작업
UNKNOWN	지원하지 않는 kind/mode

`bootstrap_gpu_server.yml`에는 실제 task가 있지만 CLI가 자동 실행하지 않는다. 운영자는 생성된 Ansible 명령의 host limit, tag와 extra variable을 검토하고 우선 `--check --diff`로 실행한다.

Kubernetes join은 token이 짧게 유효하고 잘못된 join이 scheduler와 GPU workload에 영향을 주므로 자동 기본 값이 없다. Kerberos host keytab 준비도 도메인마다 Samba/winbind 또는 SSSD/adcli 흐름이 달라 hook으로 남기고 secret 출력을 `no_log`로 숨긴다.

8. Kerberos NFS recovery 순서

기존 host에서 mount부터 실행하지 않는다. 다음 순서를 지킨다.

```
keytab 존재
-> machine kinit -k
-> NFS service kvno
-> rpc_pipefs
-> rpc-gssd active
-> mount 존재/응답 확인
-> 필요한 경우에만 gated recovery
```

읽기/계획 확인:

```
ansible-playbook server-state/ansible_playbook/kerberos_nfs_client_recovery.yml \
--limit farm8 \
--check --diff \
-e server_state_hosts=farm8
```

모든 사전조건을 확인하고 share가 실제로 missing일 때만 mount action을 명시한다.

```
ansible-playbook server-state/ansible_playbook/kerberos_nfs_client_recovery.yml \
--limit farm8 \
-e server_state_hosts=farm8 \
-e server_state_recover_mount_now=true
```

9. 새 profile 추가

1. 변경의 구현 소유 모듈을 먼저 정한다.
2. `config/profiles.yml` 에 작은 profile을 추가한다.
3. 부작용 없는 관측만 `checks` 에 둔다.
4. remediation은 가능하면 `--check --diff` 명령으로 표현한다.
5. 공유 서비스, key, join처럼 위험한 작업은 `manual` 과 reference로 남긴다.
6. 신규/기존 서버 모두 필요한 항목이면 두 profile set의 올바른 위치에 넣는다.
7. host template expansion과 set ordering unit test를 추가한다.
8. 실제 소유 모듈의 문서와 이 매뉴얼의 pipeline 표를 갱신한다.

10. 테스트

```
cd /home/jy/server_manage/server-state
python3 -m unittest discover -s tests -v
```

변경 후 추가 확인:

```
./bin/server-state --format json check \
--hosts farm8 \
--profile existing-host-drift

./bin/server-state plan \
--hosts lab10 \
--profile new-host-bootstrap
```

실제 서버 변경이 없어도 command rendering, owner, safety와 host별 경로가 맞는지 검토할 수 있어야 한다.

11. 향후 실행 모드 원칙

실제 `apply --execute` 를 추가하려면 단순히 subprocess 호출을 허용해서는 안 된다. 최소한 다음 조건이 필요하다.

- host와 profile별 명시적 승인

- check 결과와 remediation 사이의 상태 재검증
- owner module별 실행 adapter
- timeout, audit log와 secret redaction
- partial failure와 재실행의 idempotency
- manual safety 항목의 자동 실행 금지

이 조건이 설계·검토되기 전까지 dry-run 전용 상태가 안전한 기본값이다.

user-lifecycle 설계·운영 매뉴얼

원본: user-lifecycle/MANUAL.md

역할: 사용자 identity, 그룹, 포트, 컨테이너와 개인 Kerberos/NFS 준비를 하나의 생명주기로 관리한다.

1. 이 모듈이 중심인 이유

DECS 사용자 컨테이너는 Docker object 하나가 아니다. 다음 자원이 서로 같은 identity와 상태를 가져야 하는 분산 작업이다.

- 도메인별 UID MySQL의 사용자, 그룹, 예약 ID, container와 port record
- FARM/LAB target host의 Docker container
- NAS 또는 LAB storage의 사용자 home
- 선택적인 AD RFC2307 user/group과 membership
- host의 root-only user keytab과 ccache refresh timer
- 생성·삭제·연장 안내 메일, DB backup과 Excel export

`user-lifecycle` 는 이 자원의 순서와 실패 처리를 소유한다. 다른 모듈이 DB에 직접 쓰거나 container 생성 규칙을 복제하면 UID, 포트와 실제 Docker 상태가 어긋나므로 공개 `uidctl` CLI를 통과하는 것이 원칙이다.

2. 디렉터리 지도

경로	상태	핵심 기능
<code>script/uid_manager/</code>	primary	Python CLI, model, DB, service 구현
<code>script/uid_manager/services/</code>	primary	create/delete/extend/sync/cleanup/group use case
<code>script/uid_manager/kerberos/</code>	primary	AD, keytab, ccache, NFS 검증 명령과 path 생성
<code>script/ansible_playbook/</code>	primary	storage/NAS home과 host Kerberos identity 준비
<code>script/tests/</code>	primary	fake runner/repository 기반 회귀 테스트
<code>config/*.example.*</code>	template	DB, email, maintenance, Google, admin 설정 구조
<code>config/*.local.*</code>	secret/local	실제 credential과 운영 override; Git 제외
<code>server_info/servers.jsonl</code>	shared authority	FARM/LAB 서버 주소와 SSH/NAT port inventory
<code>nfs_mysql/</code>	data layer	MySQL image, schema와 server config
<code>docker-compose.yml</code>	data layer	local UID DB service/volume

경로	상태	핵심 기능
<code>maintenance/</code>	utility	사용자 삭제, email CSV 갱신 등 관리 작업
<code>ad_backup/</code>	operations	AD backup 생성·검증과 timer 설치
<code>legacy/</code>	compatibility	이전 shell 구현; 신규 기능의 기준이 아님
<code>excel_exports/</code>	generated	사용자 export 결과, source가 아님

`legacy/` 를 즉시 삭제하지 않은 이유는 기존 운영 절차의 비교·복구 근거가 필요하기 때문이다. 그러나 새 비즈니스 규칙은 Python service에 추가하고 테스트 가능하게 유지한다.

3. 코드 구조

CLI와 service 분리

`uid_manager/cli.py` 는 argument parsing, config/repository 생성과 결과 출력만 담는다. 실제 규칙은 service class가 소유한다.

CLI	Service	기능
<code>create-container</code>	<code>ContainerCreateService</code>	identity/port 결정, storage/Kerberos, Docker, DB transaction
<code>delete-container</code>	<code>ContainerDeleteService</code>	대상 단일화, Docker 삭제, DB 비활성화/port 정리
<code>extend-container</code>	<code>ContainerExtendService</code>	만료일 검색·변경과 알림
<code>expired-cleanup</code>	<code>ExpiredCleanupService</code>	만료 대상 계획 또는 실제 정리
<code>sync-containers</code>	<code>ContainerSyncService</code>	DB record와 local Docker 상태 비교·재생성 계획
<code>manage-group</code>	<code>GroupManagementService</code>	AD/DB group과 membership 관리

이 분리는 CLI 외의 timer/test에서도 동일한 규칙을 호출하고, fake repository와 recording runner로 production 접속 없이 계획을 검증하기 위한 것이다.

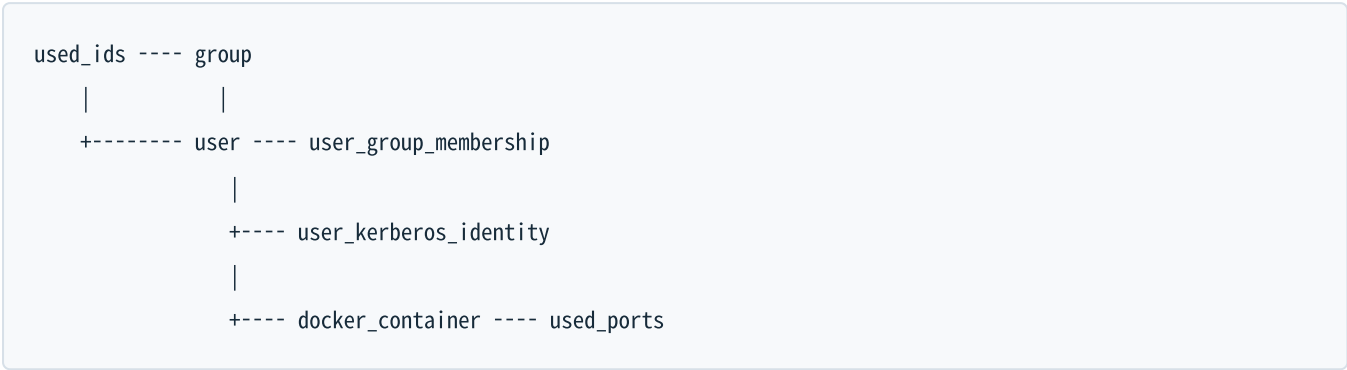
Port와 runner 추상화

`LocalRunner` 와 `AnsibleRunner` 는 subprocess와 원격 shell/playbook 호출을 한 곳에 모은다. `RecordingRunner` 는 실행 대신 command를 기록한다. `PortMapping` 은 host port, container port와 purpose를 함께 보존하여 단순 문자열 port가 SSH/Jupyter/VNC 중 무엇인지 알지 않게 한다.

`OperationPlan` 은 facts, 단계와 명령을 렌더링한다. `--dry-run` 이 실제 실행 경로와 같은 준비 로직을 통과하므로 문서용 예상 명령과 실제 입력 검증이 달라지는 것을 줄인다.

4. 데이터 모델과 권위

MySQL schema의 주요 관계:



데이터	의미
<code>used_ids</code>	UID/GID 숫자의 전역 예약
<code>group</code>	group 이름과 GID
<code>user</code>	실명, username, UID/GID, 연락처
<code>user_group_membership</code>	supplemental/primary group 관계
<code>user_kerberos_identity</code>	AD realm/SID/RFC2307과 최근 NAS/NFS 관측값
<code>docker_container</code>	image, server, 만료, 생성자와 활성 상태
<code>used_ports</code>	host port, 용도와 container record 연결

foreign key와 unique index를 쓰는 이유는 application 검증만으로 막기 어려운 중복 UID/GID, container와 port 충돌을 DB에서도 거부하기 위해서다. FARM과 LAB은 DB endpoint가 다를 수 있으므로 `AppConfig`가 domain별 host를 선택한다.

5. Container 생성 흐름

5.1 준비와 identity 채택

`ContainerCreateService.prepare()`는 이름, domain/server, 날짜, image와 port를 검증한 뒤 UID/GID source를 결정한다. 신규 사용자의 UID 우선순위는 다음과 같다.

1. DB에 기존 사용자가 있으면 DB UID/GID
2. 운영자가 명시한 `--uid / --gid`
3. Kerberos 모드에서 기존 AD RFC2307 identity
4. 기존 storage home의 숫자 owner
5. 모두 없으면 DB의 다음 사용 가능한 ID

기존 자원을 무조건 새 ID로 덮지 않고 채택하는 이유는 이미 소유권이 있는 NFS home이나 AD object를 고아로 만들지 않기 위해서다. 여러 source가 서로 다른 값을 말하면 자동 보정하지 않고 validation error로 중단한다.

port는 DB 예약과 실제 target host Docker publish port를 함께 확인한다. 기본 SSH/Jupyter, 선택적 VNC와 추가 container port를 배정하거나 `--fixed-port-mappings` 로 host:container[:purpose]를 명시할 수 있다.

5.2 실행 순서

입력 검증과 OperationPlan

|

v

target image inspect/pull

|

v

storage home + optional AD/keytab/ccache 준비

|

v

NSS/idmap/실제 NFS write 검증

|

v

docker run + inspect/port 검증

|

v

DB user/group/identity/container/port 단일 transaction

|

v

email + DB backup + export

storage와 Kerberos write 검증을 Docker/DB 확정 전에 두는 이유는 사용자가 로그인했지만 home에 쓸 수 없는 반쪽 container를 만들지 않기 위해서다. Docker 생성 뒤 DB write는 한 transaction으로 묶는다. DB 단계가 실패하면 transaction을 rollback하고 방금 만든 container 정리를 시도하여 실제 상태와 record가 갈라지는 시간을 줄인다.

`--no-db-record` 는 특수 검증용으로 Docker를 만들되 DB user/group/container/port write를 생략한다. 일반 운영 생성에는 사용하지 않는다. DB에 없는 container는 GPU user attribution, 만료 정리와 sync에서 정상 관리 대상이 아니기 때문이다.

5.3 domain별 storage

일반 LAB root_squash 경로는 storage server에서 실제 export home을 만들고 선택한 UID/GID로 owner를 설정한 뒤, target host에 mount된 `/home/tako<N>/share/user-share` 를 container `/home` 에 bind한다.

FARM은 NAS의 user-share 구조를 사용한다. Kerberos가 아닌 모드도 target host root가 NFS root_squash 때문에 직접 owner를 바꾸려 하지 않고 storage/NAS 소유 playbook을 통해 준비한다.

6. Kerberos 활성화 흐름

6.1 공통 보안 모델

```
AD user/private group + RFC2307
  |
  v
root-only keytab on target host
  |
  v
systemd refresh -> /run/user/<uid>/krb5cc
  |
  v
container receives ccache only
```

사용자 password를 DB나 container에 저장하지 않는다. AD에서 keytab을 export한 뒤 target host의 `/etc/decs-krb/keytabs/`에 mode `0400`으로 설치한다. refresh service는 ticket을 renew하거나 만료 여유가 부족하면 keytab으로 재발급하고, container에는 ccache만 전달한다.

keytab rotation은 `--rotate-kerberos-keytab`처럼 명시적으로 요청한다. 정상 container 재생성이나 만료 연장이 credential rotation을 뜻하지 않게 하기 위해서다.

6.2 FARM

FARM 흐름은 AD user/group과 RFC2307을 보장하고 NAS mapping과 home을 준비한 뒤 target host에서 해당 UID와 ccache로 실제 NFS write를 수행한다.

공유 NAS GSS/ldap service restart는 기본적으로 꺼져 있다. 새 identity의 cache 문제가 의심되더라도 NAS service 재시작은 모든 client의 GSS context를 끊을 수 있으므로 명시적 유지보수 옵션으로만 허용한다.

6.3 LAB

LAB Kerberos PoC/지원 흐름은 LAB Samba AD, storage의 전용 Kerberos export, target host의 NSS/ldap과 NFSv4.1을 함께 확인한다. storage와 client의 NFSv4 ldap domain이 다르면 owner가 `nobody`로 보이거나 `chgrp`가 실패할 수 있다. 따라서 ticket 성공만으로 완료하지 않고 숫자 owner와 실제 write를 검증한다.

6.4 DB에 남기는 Kerberos 정보

DB에는 password/keytab이 아니라 다음 비밀이 아닌 identity metadata만 둔다.

- AD username, realm, NetBIOS domain
- domain/object SID
- AD `uidNumber`, `gidNumber`
- 최근 NAS internal UID/GID
- 최근 NFS에서 본 UID/GID와 검증 시각

SID와 최근 관측값을 보존하면 같은 username이 재생성되었거나 mapping이 달라진 상황을 단순 문자열 일치보다 안전하게 발견할 수 있다.

7. 주요 CLI 사용법

설치/실행 위치:

```
cd /home/jy/server_manage/user-lifecycle/script
python3 -m pip install -e .
uidctl --help
```

생성 dry-run

```
uidctl create-container \
  --name '홍길동' \
  --username hong \
  --server-id FARM8 \
  --expiration-date 2026-12-31 \
  --image decs \
  --version cuda12.5-tf2.20-ubuntu22.04-260706 \
  --created-by admin \
  --email hong@example.org \
  --phone 000-0000-0000 \
  --dry-run
```

Kerberos/VNC는 필요할 때 명시한다.

```
uidctl create-container ... --enable-kerberos --enable-vnc --dry-run
```

삭제

DB record 검색 조건으로 대상이 하나인지 먼저 확인하고 dry-run한다.

```
uidctl delete-container \
  --server-id FARM8 \
  --container-name hong \
  --dry-run
```

만료 연장과 정리

연장은 기본적으로 계획만 만들고 실제 변경에 `--apply`가 필요하다.

```
uidctl extend-container \  
  --username hong \  
  --expiration-date 2027-06-30  
  
uidctl extend-container \  
  --username hong \  
  --expiration-date 2027-06-30 \  
  --apply
```

만료 정리도 대상 날짜와 domain을 제한해 먼저 dry-run한다.

```
uidctl expired-cleanup \  
  --today 2026-07-17 \  
  --domains FARM,LAB \  
  --dry-run
```

sync와 group

```
uidctl sync-containers --domain FARM --dry-run  
uidctl manage-group show --domain FARM --group research  
uidctl manage-group add-user \  
  --domain FARM \  
  --group research \  
  --user hong \  
  --dry-run
```

`--force`, `--auto-delete`, 실제 group delete는 영향 범위를 확인한 뒤 사용한다.

8. 설정과 secret

공개 template:

- `config/db_config.example.env`
- `config/email_config.example.env`
- `config/daily_maintenance.example.env`
- `config/google-client.example.json`
- `config/reminder_admins.example.txt`
- `ad_backup/config.example.env`

실제 값은 대응하는 `.local.*` 또는 ignored 운영 파일에 둔다. 문서, log, OperationPlan과 exception에 DB password, SMTP credential, Google secret, AD password, keytab 내용이 나타나지 않도록 command redaction을 유지한다.

`config/network_topology.json` 과 `server_info/servers.jsonl` 은 secret 저장소가 아니다. 접근 경로에 필요한 주소와 port만 넣고 password/private key는 외부 SSH/Ansible 설정이 관리한다.

9. Post action과 생성물

성공한 생명주기 작업 뒤 `PostActions` 가 이메일, DB backup과 export 도구를 호출한다. 핵심 transaction과 알림을 분리한 이유는 메일 실패가 identity/DB 일관성을 되돌리는 근거가 되지 않기 때문이다. 반대로 DB transaction이 실패했는데 성공 메일을 보내지 않도록 실행 순서는 transaction 뒤에 둔다.

`excel_exports/`, `AD_backups/`, `server_info` 의 생성된 raw 정보는 운영 artifact다. source code처럼 수정하거나 매뉴얼의 권위 있는 입력으로 삼지 않는다. 보존·암호화·삭제 주기는 별도 운영 정책에 따라야 한다.

10. 테스트

```
cd /home/jy/server_manage/user-lifecycle/script
python3 -m unittest discover -s tests -v
```

또는 프로젝트 test runner:

```
python3 tests/run_tests.py
```

중요 회귀 범위:

- 자동/고정 port 할당과 중복 거부
- DB/AD/storage 기존 identity 채택 우선순위
- create 실패 시 DB rollback과 container cleanup
- dry-run에서 원격/DB 변경이 없는지
- FARM/LAB Kerberos command와 path 차이
- group 삭제/primary 변경의 안전 gate
- sync가 DB에 없는/멈춘 container를 구분하는지
- command/log에서 password redaction

shell legacy test는 호환성 확인용이며 Python service test를 대체하지 않는다.

11. 변경 가이드

새 lifecycle command

1. request/result 또는 record를 `models.py`에 정의한다.
2. 비즈니스 순서를 `services/`의 class로 구현한다.
3. DB query는 `Repository` protocol과 `MySqlRepository`에 추가한다.
4. 외부 명령은 runner를 통하고 secret redaction을 적용한다.
5. CLI에는 parsing과 dependency 조립만 추가한다.
6. fake repository/runner를 사용한 dry-run, success와 partial failure test를 작성한다.

schema 변경

- 기존 운영 DB에 idempotent하게 적용되는 migration/ensure path를 준비한다.
- unique/foreign key와 기존 데이터 backfill 순서를 검토한다.
- FARM/LAB DB를 독립적으로 적용·검증한다.
- exporter가 읽는 column과 query 호환성을 `monitoring`에서 함께 확인한다.

Kerberos 변경

- password/keytab이 DB, plan과 log에 들어가지 않는지 확인한다.
- DB, AD, NAS/NFS와 container identity 불변 조건을 test한다.
- 공유 NAS service restart를 정상 경로에 추가하지 않는다.
- 운영 policy 변경이면 `kerberos-nfs/APPLIED_STATE.md`와 runbook도 갱신한다.

12. 운영 안전 수칙

- 모든 destructive 명령은 domain, server와 대상 container가 하나인지 확인한다.
- 실제 생성 전 `--dry-run`의 identity source, mount, image와 port를 검토한다.
- 기존 AD 또는 storage identity와 요청 UID/GID 충돌을 강제로 덮지 않는다.
- DB와 실제 Docker 상태가 어긋나면 원인을 확인하고 `sync --auto-delete`를 바로 사용하지 않는다.
- keytab과 사용자 password를 container나 backup export에 포함하지 않는다.
- `legacy/`를 수정해 신규 규칙을 우회하지 않는다.
- DB 없는 test container는 이름, 수명과 정리 책임을 명확히 한다.